

İleri Ağaçlar

CEN207 Veri Yapıları — Hafta 11

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	BST: ekleme, arama, silme, denge neden önemli
2	AVL (4 döndürme), kırmızı-siyah, splay
3	2-3 ağacı, segment ağacı, Fenwick ağacı, teknik seçme

Öğrenme çıktıları: ÖÇ.1 (temel veri yapılarını açıklama) · ÖÇ.2 (karmaşıklığı analiz etme) · ÖÇ.7 (doğru yapıyı seçme)

Bu haftanın konuları — nerede

Konu	Nerede
BST ekleme/arama/silme, dejenere durum	Bölüm 1
AVL, kırmızı-siyah, splay, 2-3 ağacı	Bölüm 2–5
Segment ağacı, Fenwick ağacı	Bölüm 6–7
Teknik seçme	Bölüm 8

Kod örnekleri nasıl çalışır

- Her fikrin tam bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-11/c/` ve `code/week-11/java/`
- Her programın beklenen çıktısı hafta notlarında

Hafta 4'ten köprü

- Hafta 4 size **ağaç kelime dağarcığını** verdi: kök, yaprak, derinlik, yükseklik, alt ağaç
- Hafta 4 size **dolaşmaları** verdi: inorder, preorder, postorder, level-order
- Hafta 4'ün öbeği sol-sağ hiç **sıralı** değildi, yalnızca ebeveyn-çocuk
- Bugün ilk kez bir sıralama kuralı ekliyoruz

Tekrar — Hafta 4: yükseklik ve derinlik

- Bir düğümün **derinliği** = kökten ona kaç kenar (kök = 0)
- Bir ağacın **yüksekliği** = herhangi bir düğümün en büyük derinliği
- Boş ağaç: gelenek gereği yükseklik -1; tek düğüm: yükseklik 0
- Bugün: yükseklik, her maliyeti belirleyen tek sayı

Haftanın haritası — tek kural

- Bir **ikili arama ağacı**: sol alt ağaç küçük, sağ alt ağaç büyük
- Bu kural tek başına hızlı arama verir — AĞAÇ SIĞ KALIRSA
- Dört farklı strateji onu sığ tutar: AVL, kırmızı-siyah, splay, 2-3
- İki ağaç daha, tek-anahtar yerine **aralık** sorularını yanıtlar

Haftanın haritası — genel bakış

Dengeli BST'ler	Aralık sorgusu ağaçları
AVL — katı denge çarpanı	Segment ağacı — bir kur, $O(\log n)$ 'de sorgula
Kırmızı-siyah — daha gevşek, renk tabanlı	Fenwick ağacı — tek dizi, i & $-i$
Splay — denge yok, kullanıma uyum sağlar	—
2-3 ağacı — yalnız kökte büyür	—

Tekrar — Hafta 4: dolaşmalar

- **Inorder**: sol, ziyaret, sağ — bir BST'de sıralı düzen
- **Preorder**: ziyaret, sol, sağ — bir ağacın biçimini kopyalar
- **Postorder**: sol, sağ, ziyaret — bir ağacı silmek için güvenli
- **Level-order**: açık bir kuyruk, genişlik öncelikli

Kelime dağarcığı kontrolü

Terim	Anlamı
Denge çarpanı (balance factor)	$\text{height}(\text{sol}) - \text{height}(\text{sağ})$
Döndürme (rotation)	$O(1)$ yerel gösterici düzenlemesi
Amortize maliyet	uzun bir işlem dizisi üzerinde ortalananmış
Tersinir işlem	çıkarmayla geri alınabilir (toplam, min değil)

1. İkili Arama Ağacı

Başlangıç sorusu

Hafta 1: sıralı dizi, ikili arama — hızlı, ama ekleme $O(n)$ tutar.

Hafta 2: bağlı liste — ekleme $O(1)$, ama arama $O(n)$ ister.

İKİSİNİ DE $O(n)$ 'den daha iyi yapan bir yapı var mı?

Kısa bir tarihçe

- BST fikirleri **1959–1962** arasında bağımsız olarak ortaya çıkar
- P. F. Windley, A. D. Booth & A. J. T. Colin, T. N. Hibbard
- **Hibbard, 1962** — genellikle silmeyi çözmesiyle anılır
- Silme, tam olarak bölüm 1.4'ün bugün ele aldığı şeydir

Sezgi — bir telefon rehberi, ama ağaç

- Orta sayfayı açın: adınız ondan önce mi sonra mı?
- Yarılamaya devam edin — bu, bir dizide ikili aramadır
- Bir BST, aynı yarılamayı yapının **biçimine** pişirir
- Her düğümde kural: sol küçük, sağ büyük

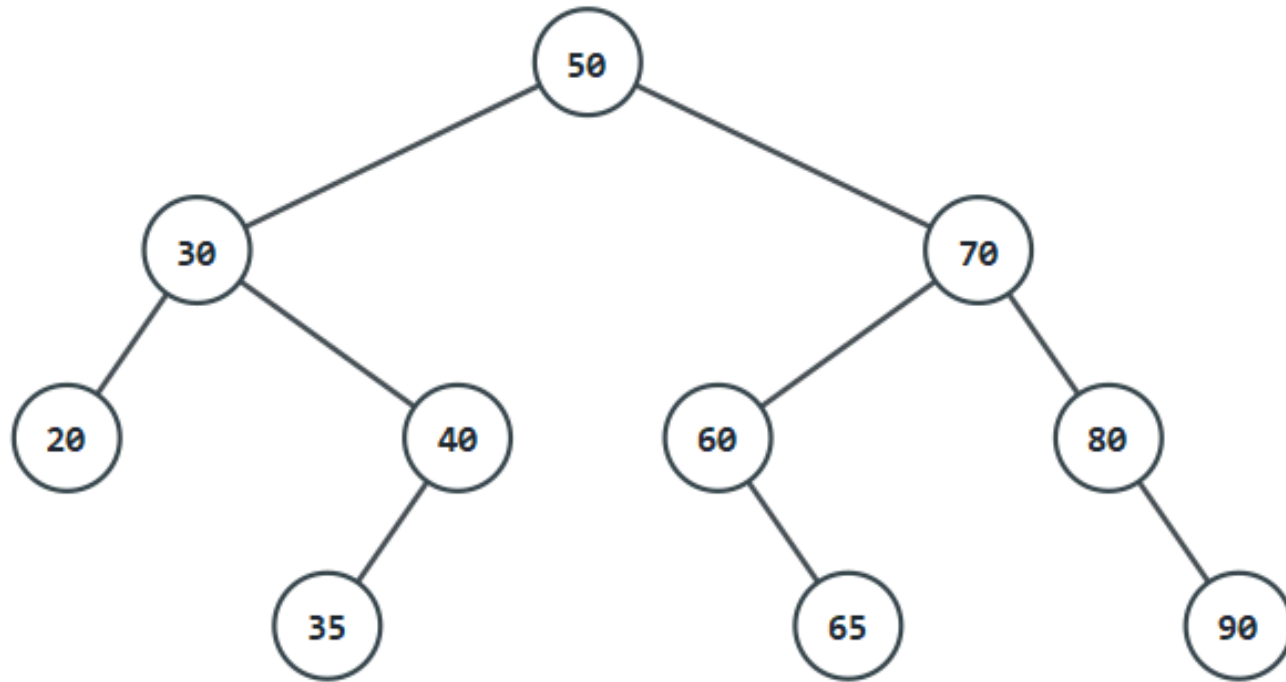
BST soyut veri türü

İşlem	Ne yapar	Karmaşıklık
<code>insert(key)</code>	Anahtarı ekler; yinelenen yoksayılır	$O(h)$
<code>search(key)</code>	Var olup olmadığını bildirir	$O(h)$
<code>delete(key)</code>	Varsa anahtarı kaldırır	$O(h)$
<code>min / max</code>	En küçük / en büyük anahtar	$O(h)$

Bellekte — ekleme fikri

- Kökten aşağı yürü, her düğümde `key` 'i karşılaştı
- Küçük → sola git; büyük → sağa git; eşit → yinelenen, dur
- Bir `NULL` çocuğa ulaş → yeni düğümün yeri orası
- Bağla; ağaçta başka hiçbir şey hareket etmez

İkili arama ağacı: ekleme



eklenecek:

50	30	70	20	40
60	80	35	65	90

Bitti: 10 anahtar işlendi. Her ekleme, ağacın yüksekliği (height) kadar karşılaştırma yapar — dengeli bir ağaçta bu $O(\log n)$, ama sıra kötü olursa (bak: bst-degenerate) $O(n)$ 'e kadar çıkabilir.

Kod — bst_insert()

```
Node *bst_insert(Node *root, int key) {  
    Node *cur = root, *parent = NULL;  
    while (cur != NULL) {  
        parent = cur;  
        if (key == cur->key) return root;  
        if (key < cur->key) cur = cur->left;  
        else cur = cur->right;  
    }  
    Node *n = malloc(sizeof(Node));  
    n->key = key; n->left = NULL; n->right = NULL;
```

Kod — bst_insert(), bağlama

```
if (parent == NULL) return n;  
if (key < parent->key) parent->left = n;  
else parent->right = n;  
return root;  
}
```

Beklenen çıktı

```
insert(50): inorder = 50  height = 0  
insert(30): inorder = 30 50  height = 1  
insert(70): inorder = 30 50 70  height = 1  
insert(20): inorder = 20 30 50 70  height = 2
```

insert neden $O(h)$

- Ziyaret edilen seviye başına en fazla bir karşılaştırma
- Geçilen bir düğümü asla tekrar ziyaret etmez
- h küçük (dengeli) $\rightarrow$ hızlı; h büyük (zincir) $\rightarrow$ yavaş
- Bölüm 1.4, h 'nin ne kadar büyüyebileceğini tam olarak gösterir

Sık hata

- `root = bst_insert(root, key);` yazmayı unutmak
- Yeniden atama olmadan, çağırının `root` 'u asla güncellenmez
- C/Java burada göstericileri/referansları **değer olarak** geçirir
- Fonksiyon (olası yeni) kökü **döndürmelidir**

Mini soru

Animasyonun "artan sıra" uç durumunda, her yeni anahtar SAĞ çocuk olur. Neden hiç sol olmaz?

Mini yanıt

- Sonraki her anahtar, zaten eklenmiş her şeyden büyüktür
- Bu yüzden yürüyüş sırasında her karşılaştırma "sağa git" der
- Ağaç tamamen sağa yaslanır — bir zincir
- Bu tam olarak bölüm 1.4'ün konusu, hemen ardından

Başlangıç sorusu — arama

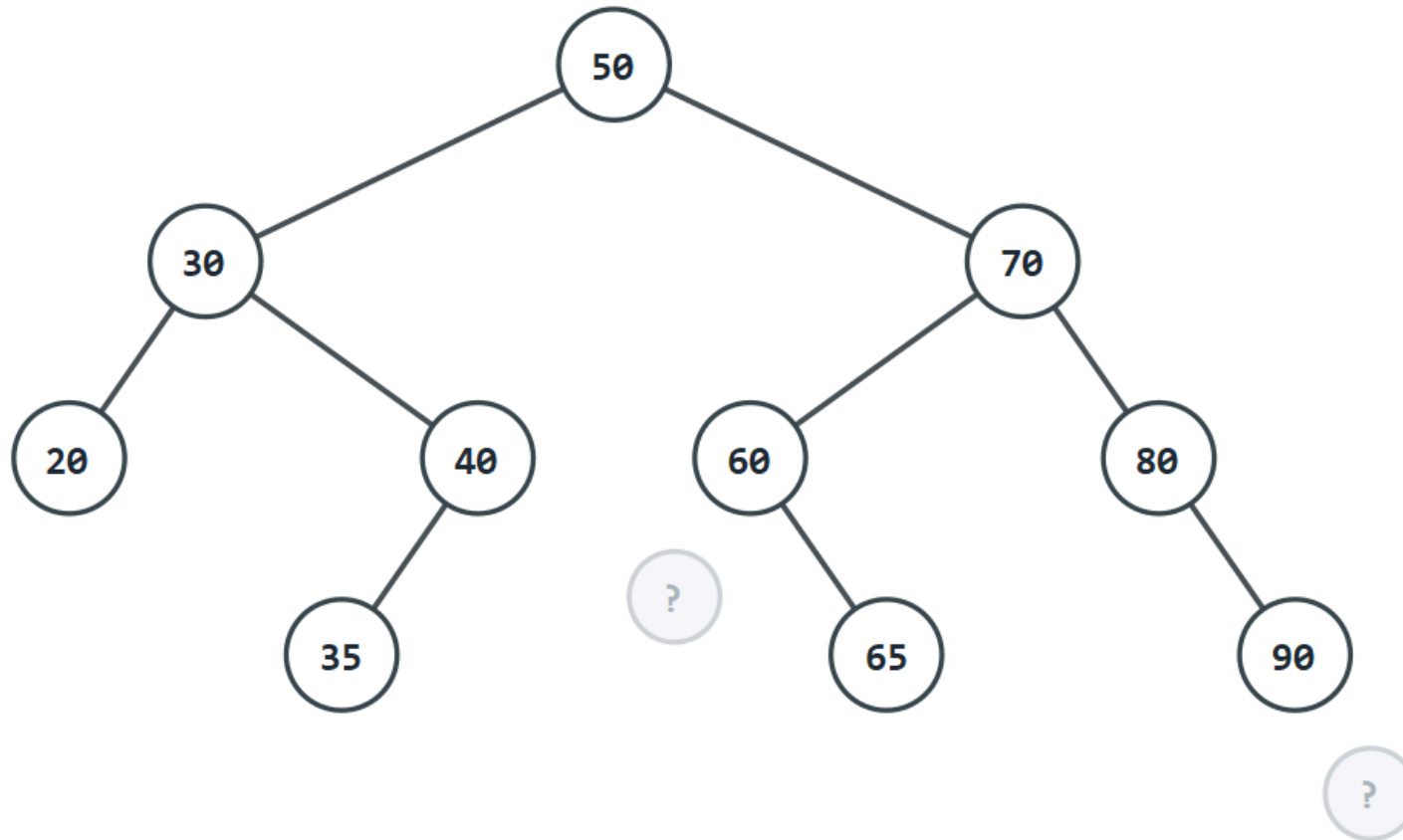
`insert` zaten anahtarları karşılaştırarak aşağı yürüyor.

`search` neredeyse aynı yürüyüş — en kötü durumda kaç düğüm?

Fikir — arama

- Eşit → bulundu; küçük → yalnızca sol alt ağaçta olabilir
- Sıralama kuralı bunu GARANTİ EDER, sağ alt ağaç olamaz
- Büyük → ayna görüntüsü
- Bir eşleşmeden önce ulaşılan `NULL` → yok

İkili arama ağacı: arama



Bitti: 5 arama tamamlandı. Her arama en fazla ağacın yüksekliği kadar karşılaştırma yapar.

Kod — bst_search()

```
int bst_search(Node *root, int key) {  
    Node *cur = root;  
    probes = 0;  
    while (cur != NULL) {  
        probes++;  
        if (key == cur->key) return 1;  
        if (key < cur->key) cur = cur->left;  
        else cur = cur->right;  
    }  
    return 0;  
}
```

Beklenen çıktı

```
search(65): found, 4 probes  
search(20): found, 3 probes  
search(55): not found, 3 probes  
search(100): not found, 4 probes
```

search neden $O(h)$

- insert ile aynı seviye-başına-bir-karşılaştırma argümanı
- En iyi durum $O(1)$: kökün kendisi
- En kötü durum: ağaç ne kadar derinse
- Yine: her şey h 'ye bağlı

Sık hata

- "Bulunamadı"yı bir hata koşulu olarak ele almak
- Bu normal, beklenen bir sonuçtur, çökme değil
- `while (cur != NULL)` koruyucusu bunu temiz ele alır
- Bir eşleşmeden sonra karşılaştırmaya devam etmek de israf

Mini soru

10 düğümlü artan-eklemeli bir zincirde `search(10)` 10 prob tutar. `search(1)` yalnız 1 tutar. Neden bu kadar fark?

Mini yanıt

- O ağaç saf bir zincirdir: 1 kökte, 10 en derin yaprakta
- Kökün kendi anahtarı: tek karşılaştırma
- En derin yaprak: ona kadar her seviye için bir karşılaştırma
- En kötü durumda yükseklik + 1 karşılaştırma

Başlangıç sorusu — silme

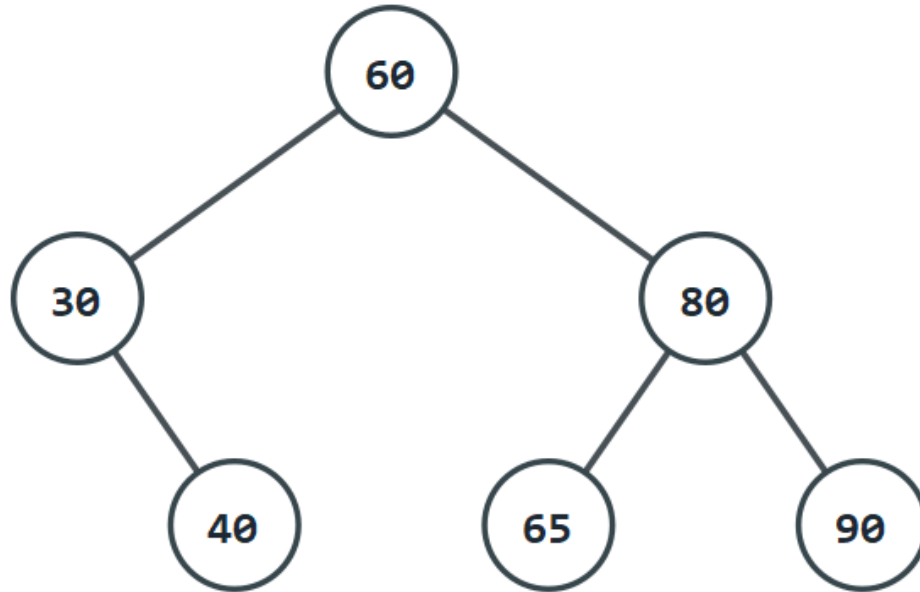
Bir **yaprağı** silmek kolaydır: ayırın.

İki çocuklu bir düğümü silmek öyle değil. Neden?

Fikir — üç durum

Durum	Düzeltilme
Yaprak	Ayırın
Tek çocuk	Ebeveyn doğrudan çocuğa bağlanır
İki çocuk	Ardılın anahtarını kopyalayın, sonra ONU silin

İkili arama ağacı: silme



Bitti: 4 silme işlendi. Kalan ağaç hâlâ geçerli bir BST.

Kod — bst_delete(), düğümü bulma

```
Node *bst_delete(Node *root, int key) {  
    Node *cur = root, *parent = NULL;  
    while (cur != NULL && key != cur->key) {  
        parent = cur;  
        cur = (key < cur->key) ? cur->left : cur->right;  
    }  
    if (cur == NULL) return root;
```

Kod — bst_delete(), iki çocuk

```
if (cur->left != NULL && cur->right != NULL) {  
    Node *succ = cur->right, *succParent = cur;  
    while (succ->left != NULL) {  
        succParent = succ; succ = succ->left;  
    }  
    cur->key = succ->key;  
    parent = succParent; cur = succ;  
}
```

Beklenen çıktı

```
delete(35): inorder = 20 30 40 50 60 65 70 80 90  
delete(70): inorder = 20 30 40 50 60 65 80 90  
delete(50): inorder = 20 30 40 60 65 80 90  
delete(20): inorder = 30 40 60 65 80 90
```

delete neden $O(h)$

- Düğümü bulmak: $O(h)$
- Bir ardıl bulmak: en fazla bir $O(h)$ daha
- Arama yolundaki düğümleri asla yeniden ziyaret etmez
- insert ve search ile aynı en kötü durum

Sık hata

- Ardılın anahtarını kopyalayıp ONUN düğümünü silmeyi unutmak
- Anahtar artık ağaçta iki kez var
- Ardıl ile öncel: ikisi de çalışır, ama tutarlı kalın
- C'de `free()` unutmak — gerçek bir bellek sızıntısı

Mini soru

Bir BST silmenin ardılı neden EN FAZLA bir çocuğa sahip olmayı garantiler?

Mini yanıt

- Ardıl = sağ alt ağacın en soldaki düğümü
- En solda olmak, tanım gereği sol çocuğu olmaması demek
- Bir sağ çocuğu olabilir ya da olmayabilir
- Asla ikisi birden — bu yüzden hep durum 1 ya da 2

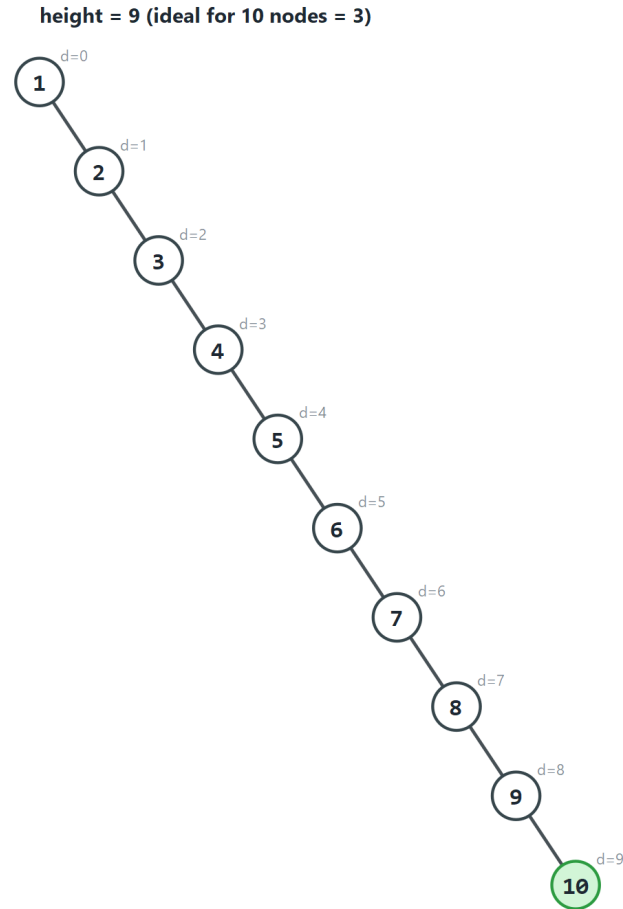
Başlangıç sorusu — dejenere

Şimdiye kadarki her bölüm maliyeti $O(h)$ olarak belirtti.
 n anahtar rastgele bir sırayla, h SOMUT OLARAK nedir?

Fikir — küme değil, sıra

- Her teknik DEĞERLERİ karşılaştırır, ağacın BİÇİMİNE hiç bakmaz
- Sıralı girdi: her yeni anahtar şimdiye kadarki her şeyden büyük
- Her yeni anahtar sonuncudan bir seviye daha derine eklenir
- Ağaç bir zincire **dejenere olur**: yükseklik $n - 1$

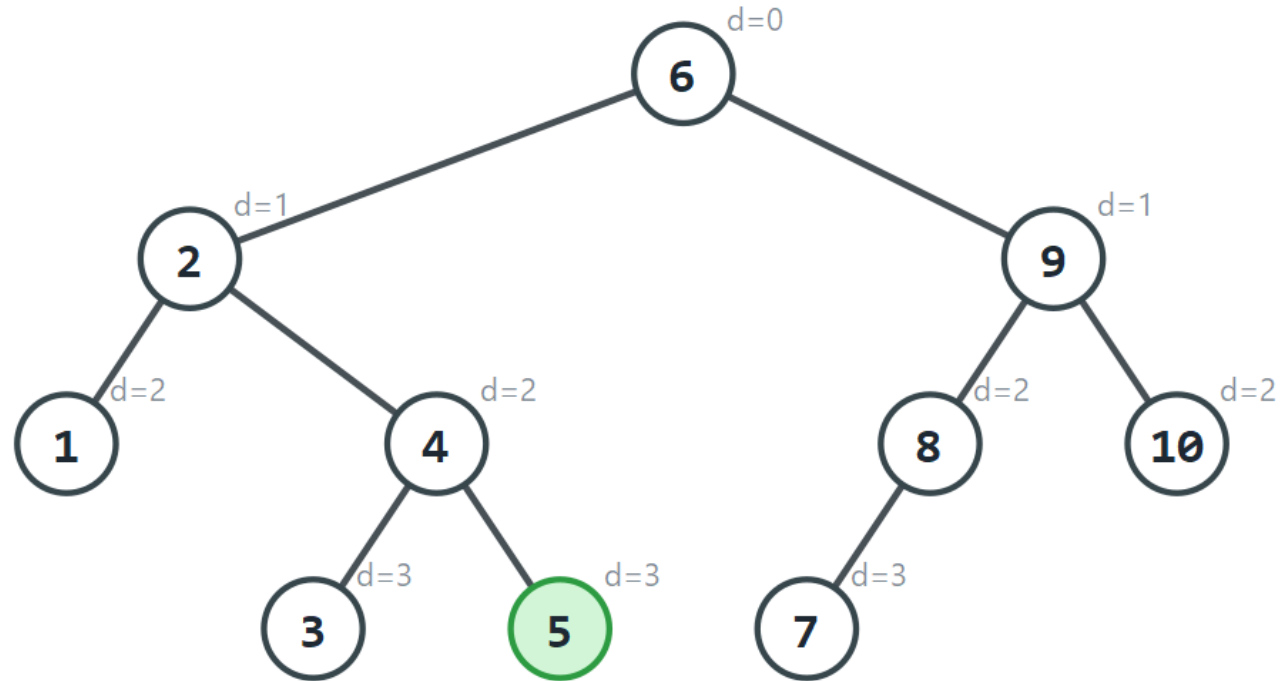
Denge neden önemli



Bitti. 10 düğüm, yükseklik = 9; dengeli bir ağaçta ideal yükseklik yalnızca 3 olurdu (yaklaşık 3.0x fark). Bu yüzden AVL, kırmızı-siyah gibi DENGELİ ağaçlar var — sonraki animasyonlarda.

Aynı anahtarlar, karıştırılmış

height = 3 (ideal for 10 nodes = 3)



Bitti. 10 düğüm, yükseklik = 3; dengeli bir ağaçta ideal yükseklik yalnızca 3 olurdu (yaklaşık 1.0x fark). Bu yüzden AVL, kırmızı-siyah gibi DENGELİ ağaçlar var — sonraki animasyonlarda.

Beklenen çıktı

```
insert(10): height = 9  (ideal for 10 nodes = 3)  
final: n = 10, height = 9, ideal = 3
```

kariştirilmiş uç durumuna karşı: `final: n = 10, height = 3, ideal = 3`

Bunun karmaşıklık için anlamı

- En kötü durum yüksekliği: $n - 1$ (sıralı ya da ters sıralı girdi)
- Bölüm 1.1–1.4'ün her işlemi $O(n)$ olur
- Ortalama durum (rastgele sıra): $O(\log n)$ — ama "rastgele" garantili değil
- Sıralı girdi pratikte yaygın: içe aktarmalar, tekrar oynatılan günlükler

Sık hata

- "BST" olmasının otomatik olarak $O(\log n)$ demek olduğunu varsaymak
- Yalnızca rastgele test verisiyle kıyaslama yapmak
- Rastgele veri tam olarak bu başarısızlık biçimini gizler
- Yalnızca KENDİNİ DENGELİYEN bir BST en kötü durumda $O(\log n)$ garanti eder

Mini soru

Zaten sıralı olduğunu bildiğiniz veriden bir BST kurmalısınız.
Ağaç türünü değiştirmeden en ucuz çözüm?

Mini yanıt

- Eklemekten önce anahtarları rastgele bir sıraya karıştırın
- Ya da: orta elemanı kök seçin, her iki yarıda özyineleyin
- Doğrudan $O(n)$ 'de mükemmel dengeli bir ağaç kurar, döndürme yok
- Bölüm 2–5, GENEL problemi otomatik olarak çözer

2. AVL Ağaçları

Başlangıç sorusu

İşlemler hangi sırayla gelirse gelsin sığ kalan bir BST'ye ihtiyacımız var. Böyle bir şey var mı?

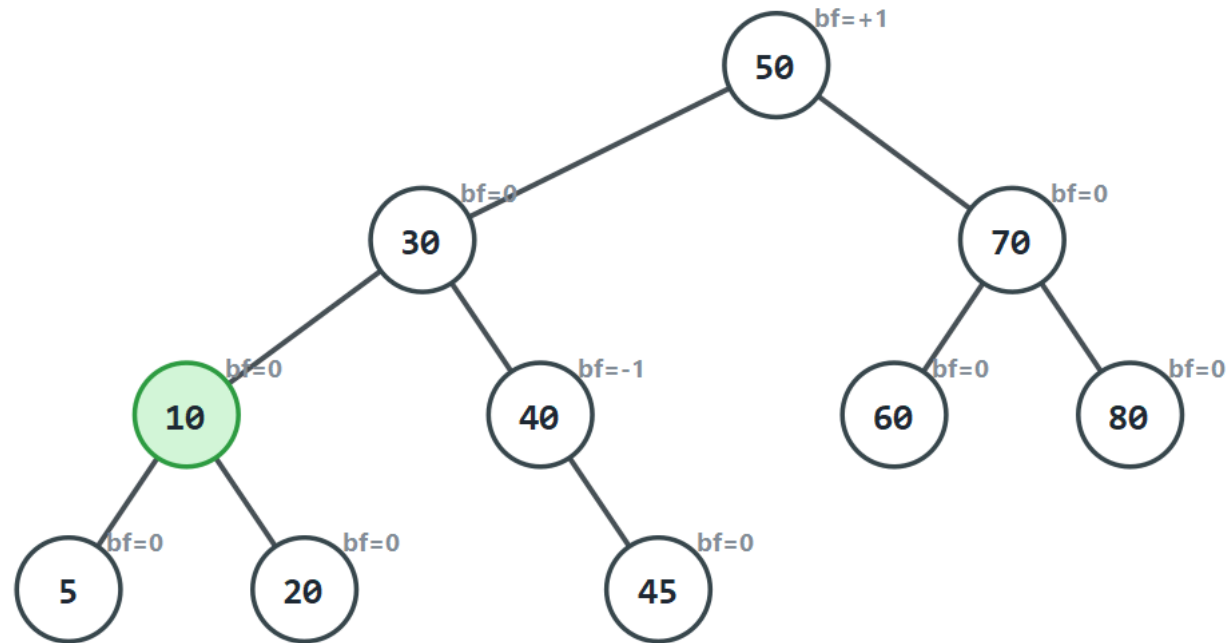
Kısa bir tarihçe

- **1962** — Georgy Adelson-Velsky & Evgenii Landis
- "AVL" = baş harfleri
- Yayımlanan ilk kendini dengeleyen ikili arama ağacı
- Fikir: her düğümde bir **denge çarpanı** izle

Dört döndürme durumu

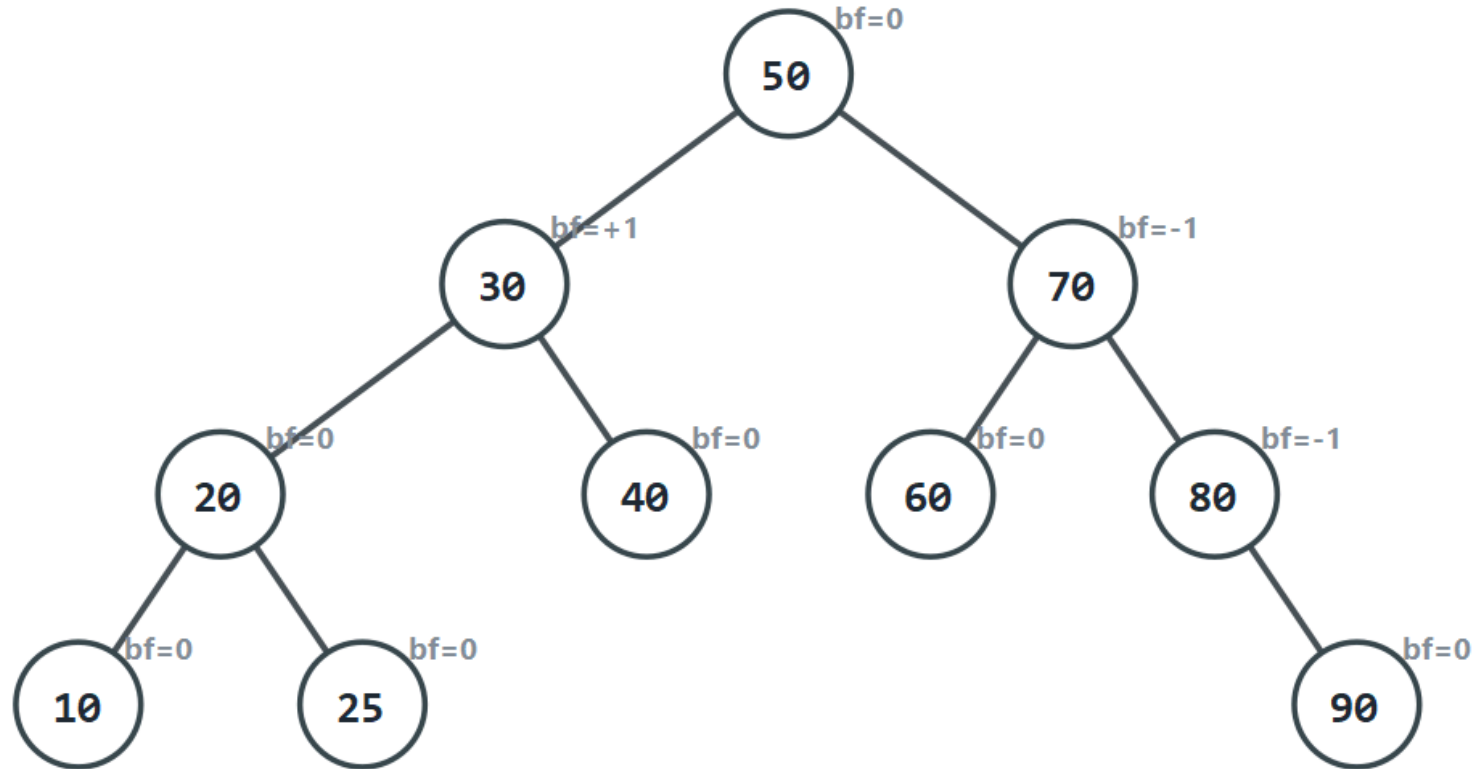
Durum	Biçim	Düzeltilme
LL	sol-ağır, sol çocuk sol-ağır	tek sağa döndürme
RR	sağ-ağır, sağ çocuk sağ-ağır	tek sola döndürme
LR	sol-ağır, sol çocuk sağ-ağır	sol çocuğu sola, sonra sağa
RL	sağ-ağır, sağ çocuk sol-ağır	sağ çocuğu sağa, sonra sola

AVL: dört dengeleme durumu



Bitti. Bir ekleden sonra AVL'de en fazla BİR döndürme (LL/RR) ya da BİR çift döndürme (LR/RL) gerekir — asla daha fazlası; bu yüzden insert hâlâ $O(\log n)$.

Döndürme gerekmiyor



Bitti. Bir eklemekten sonra AVL'de en fazla BİR döndürme (LL/RR) ya da BİR çift döndürme (LR/RL) gerekir — asla daha fazlası; bu yüzden insert hâlâ $O(\log n)$.

Mini soru

Bir eklemekten sonra, bir AVL ağacı en fazla kaç döndürmeye ihtiyaç duyabilir?

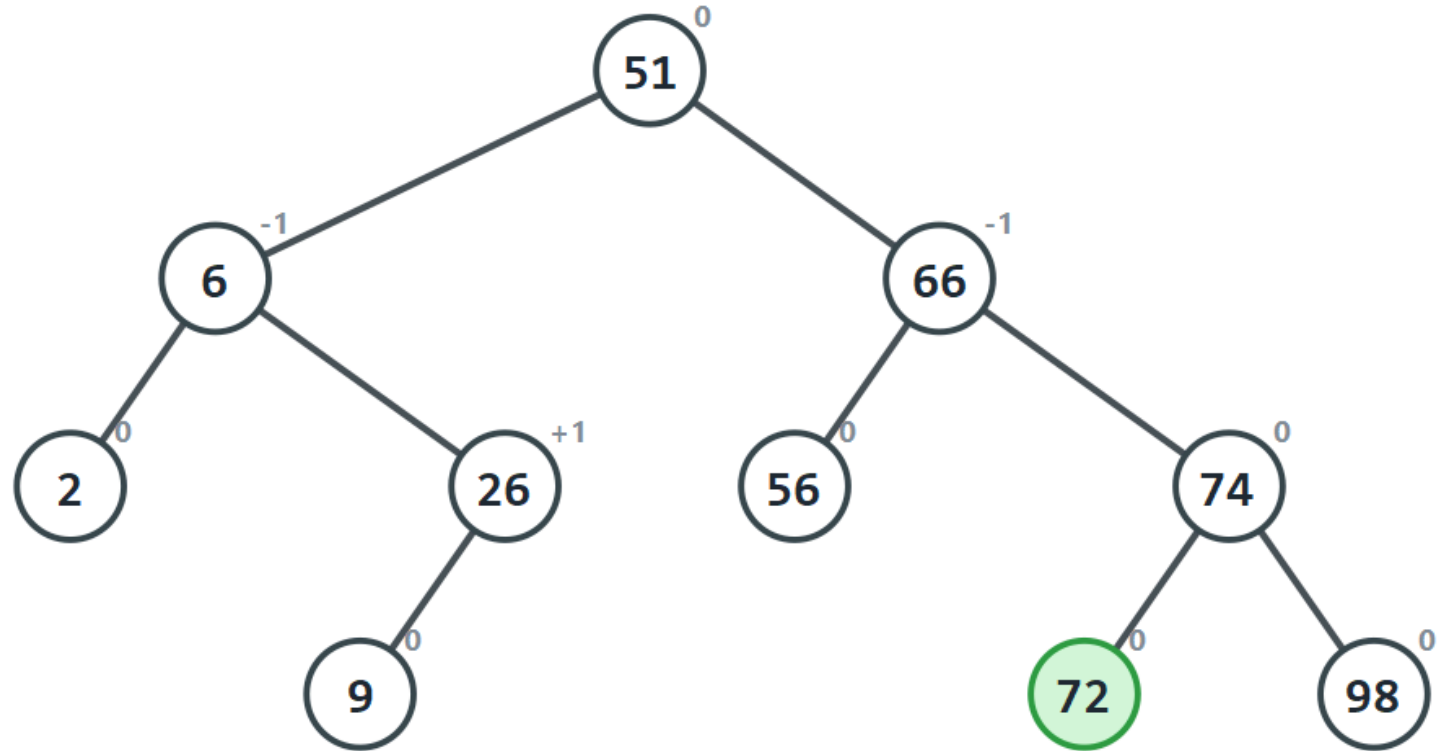
Mini yanıt

- En fazla BİR tekli döndürme, ya da BİR çift döndürme
- Düzeltme, alt ağacın eklemekten ÖNCEKİ yüksekliğini tam kurar
- Bu yüzden daha yukarıdaki hiçbir ata dengesiz olamaz
- Ekleme asla yukarı doğru yayılmak zorunda kalmaz

AVL ekleme — fikir

- Sıradan özyinelemeli BST ekleme
- Çağrı yığını geri sarılırken: `update_height` , sonra `rebalance`
- Kontrol, özyineleme geri sarılırken HER seviyede olur
- Bulunan ilk (ve tek) dengesiz düğüm hemen düzeltilir

AVL ağacı: ekleme



Bitti: 10 anahtar eklendi, ağaç her adımda dengeli kaldı. insert her zaman $O(\log n)$: karşılaştırmalar artı en fazla bir (çift) döndürme.

Kod — rebalance()

```
Node *rebalance(Node *n) {  
    int bf = height(n->left) - height(n->right);  
    if (bf > 1 && height(n->left->left)  
        >= height(n->left->right))  
        return rotate_right(n);  
    if (bf > 1) {  
        n->left = rotate_left(n->left);  
        return rotate_right(n);  
    }  
}
```

Beklenen çıktı — artan anahtarlar

```
insert(10): height = 3
```

Bölüm 1.4'ün düz BST'si AYNI 10 artan anahtar için **yükseklik 9'a** ulaştı.

AVL neden $O(\log n)$

- Yükseklik asla $1.44 \cdot \log_2(n + 2)$ 'yi geçmez
- Her işlem $O(\log n)$ — ORTALAMA değil, EN KÖTÜ durumda da
- insert: en fazla bir (çift) döndürme
- delete: $O(\log n)$ 'ye kadar döndürme, ama her biri $O(1)$

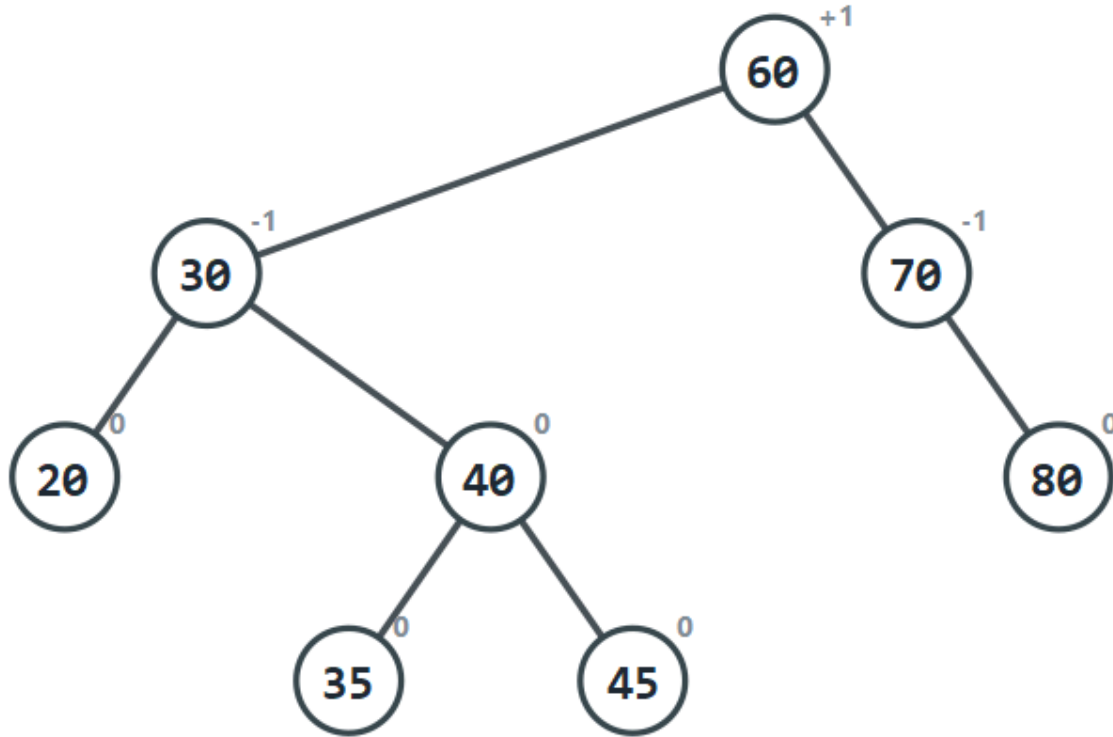
Sık hata

- Denge çarpanını kontrol etmeden önce `update_height` 'ı unutmak
- `rebalance` sonra BAYAT bir yükseklik okur
- LL'yi mi LR'yi mi az önce eklenen anahtara bakarak seçmek (silme için bozulur)
- Denge-çarpanı tabanlı karar hem ekleme hem silme için çalışır

AVL silme — yeniden dengeleme katlanabilir

- Bölüm 1.4'ün ayırma mantığını (yaprak/tek/iki çocuk) birebir kullanır
- Fark: `rebalance` HER atada çalışır, yalnız ilkinde değil
- Bir silme bir alt ağacın yüksekliğini küçültebilir
- O küçülme köke kadar yayılmaya devam edebilir

AVL ağacı: silme



Bitti: 4 silme işlendi. Ağaç her zaman AVL kaldı (bf hep -1..+1). delete yine $O(\log n)$, ama tek eklemekten farklı olarak birden fazla seviyede döndürme gerekebilir.

Beklenen çıktı

```
delete(90): height = 3, inorder = 10 20 25 30 ...  
delete(45): height = 3, inorder = 10 20 25 30 ...
```

Altı silme boyunca yükseklik 3'te kalır — hep yeniden dengelenir.

AVL silme neden hâlâ $O(\log n)$

- $O(\log n)$ 'ye kadar döndürme — en kötü durumda ata-seviyesi başına bir
- Her tek döndürme yine $O(1)$
- $O(\log n)$ döndürme $\times$ her biri $O(1)$ = toplamda $O(\log n)$
- Ekleme ile aynı asimptotik sınır, yalnız sabiti daha büyük

Sık hata

- Silmenin de, ekleme gibi, yalnız bir döndürme gerektireceğini varsaymak
- "Zor" senaryo tam olarak bunu çürütmek için kurulmuştur
- C'de ayrılan düğümü `free()` etmeyi unutmak (sızıntı)
- "Bütün anahtarları boşalana kadar sil"i açıkça test etmemek

Mini soru

h yüksekliğinde bir AVL ağacının en az kaç düğümü vardır?

Mini yanıt

- $N(h) = 1 + N(h-1) + N(h-2)$ — Fibonacci yinelemesi
- $N(h)$, h 'de ÜSTEL olarak büyür
- Bu yüzden h , n 'de yalnızca LOGARİTMİK olarak büyür
- Kanıtın tamamı, tek satırda budur

3. Kırmızı-Siyah Ağaçlar

Başlangıç sorusu

AVL'nin katı denge çarpanı neredeyse her eklemede yeniden dengeleme isteyebilir. Daha gevşek bir kural var mı?

Kısa bir tarihçe

- **1972** — Rudolf Bayer: "simetrik ikili B-ağaçları"
- **1978** — Guibas & Sedgewick: "kırmızı-siyah" adı
- Modern ekleme algoritması 1978'e dayanır
- Tam yükseklik karşılaştırması yerine dört basit, YEREL kural

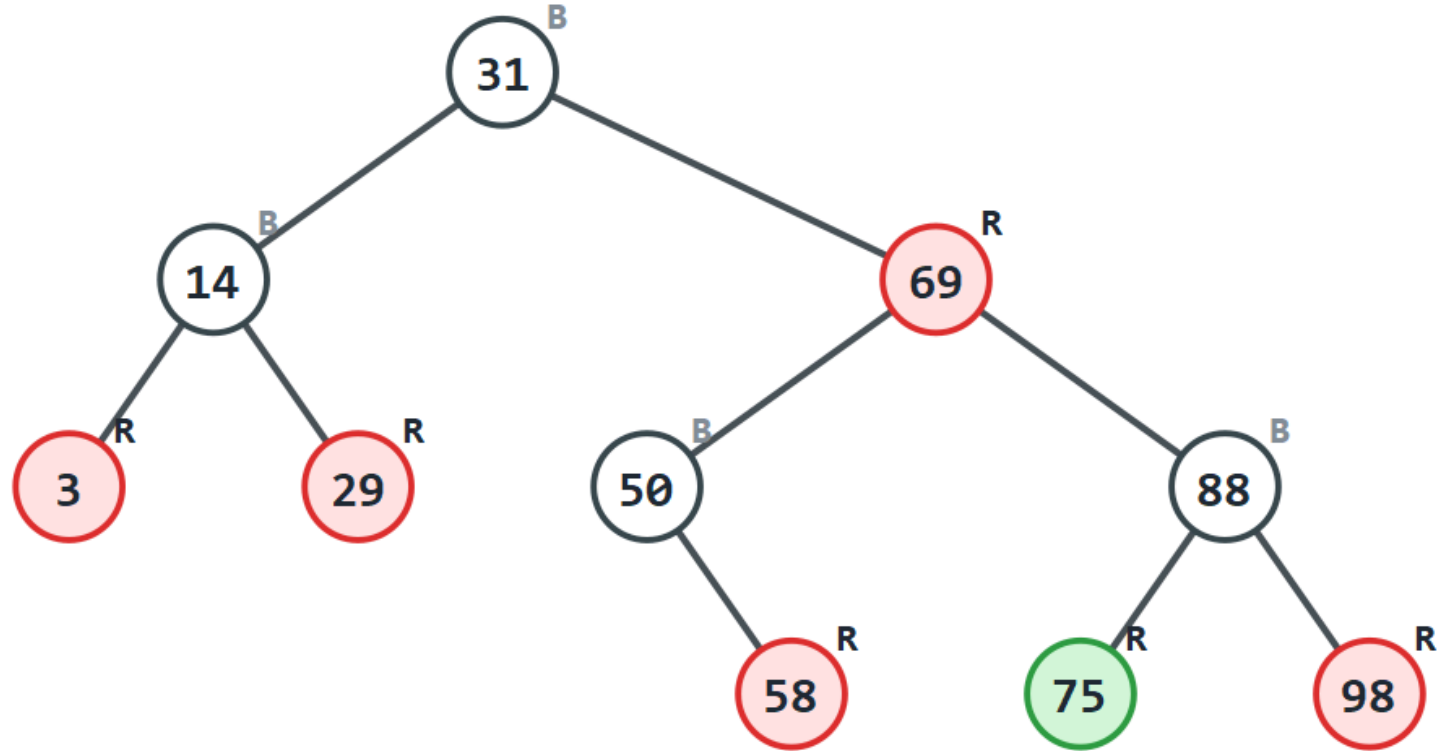
Dört kural

- Her düğüm **kırmızı** ya da **siyahtır**
- Kök her zaman **siyahtır**
- Kırmızı bir düğümün asla kırmızı çocuğu olmaz ("art arda iki kırmızı yok")
- Her kök-NULL yolu aynı **siyah-yükseklığe** sahiptir

Üç düzeltme (fixup) durumu

Durum	Durum	Düzeltilme
1	Amca KIRMIZI	ebeveyn+amca'yı siyaha, büyükanne/babayı kırmızıya, yukarı devam
2	Amca siyah, "üçgen"	ebeveyni döndür, durum 3'e indirger
3	Amca siyah, "doğrusal"	büyükanne/babayı döndür + yeniden renklendir, bitti

Kırmızı-siyah ağaç: ekleme



Bitti: 10 anahtar eklendi. Her fixup, en fazla $O(\log n)$ adım kırmızı-kırmızı ihlalini yukarı taşır, sonra en fazla bir (tek ya da çift) döndürmeyle biter — insert yine $O(\log n)$.

Kod — fixup(), durum 1

```
while (z->parent && z->parent->color == RED) {  
    Node *p = z->parent, *g = p->parent;  
    Node *u = (p == g->left) ? g->right  
                : g->left;  
    if (u && u->color == RED) {  
        p->color = BLACK; u->color = BLACK;  
        g->color = RED; z = g; continue;  
    }  
}
```

Beklenen çıktı

```
insert(3): root = 3, bh = 1, inorder = 3B  
insert(69): root = 3, bh = 1, inorder = 3B 69R  
insert(31): root = 31, bh = 1, inorder = 3R 31B 69R
```

Kırmızı-siyah neden $O(\log n)$

- Yükseklik asla $2 \cdot \log_2(n + 1)$ 'i geçmez
- Kanıt: hiçbir yol en kisasının iki katından fazla olamaz
- (kırmızı düğümler asla bitişik olamaz)
- AVL'den biraz daha gevşek sınır, pratikte daha az döndürme

Sık hata

- fixup döngüsü bittikten sonra kural 2'yi unutmak
- Durum 1, ihlali köke kadar taşıyabilir
- Sondaki koşulsuz `root->color = BLACK;` isteğe bağlı DEĞİL
- "Amca"yı yeni düğümün kendi kardeşiyle karıştırmak

Mini soru

KIRMIZI bir yaprak eklemek kural 4'ü (eşit siyah-yükseklik) neden asla bozamaz, yalnız kural 3'ü bozabilir?

Mini yanıt

- Kırmızı bir yaprak herhangi bir yolun siyah-düğüm sayısına 0 katkıda bulunur
- Her kök-yaprak siyah-yüksekliği tam olarak ne idiyse öyle kalır
- Yalnız kural 3 ("iki kırmızı yok") bozulabilir, ve yalnız ebeveyn kırmızıysa
- Bu tam olarak fixup'ın onarmak için tasarlandığı durumdur

4. Splay Ağaçları

Başlangıç sorusu

AVL ve kırmızı-siyah, HER düğümde, HER işlem için defter tutma maliyeti öder — nadiren dokunulan anahtarlar için bile. Farklı bir yol?

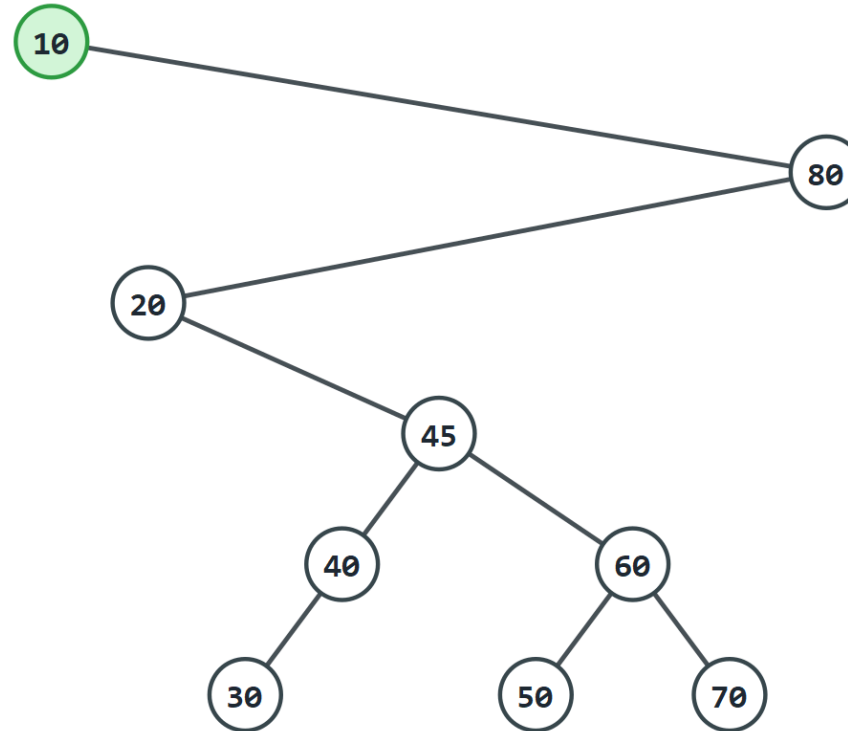
Kısa bir tarihçe

- **1985** — Daniel Sleator & Robert Tarjan
- "Kendini ayarlayan ikili arama ağaçları"
- Hiç denge bilgisi tutulmaz — düğüm başına sıfır ekstra bellek
- Bunun yerine: her erişim ağacı yeniden biçimlendirir

Fikir — köke taşı

Hareket	Ne zaman	Ne olur
zig	ebeveyn kök	tek bir döndürme
zig-zig	düğüm & ebeveyn ikisi de sol (ya da sağ) çocuk	önce ebeveyn, sonra düğüm döner
zig-zag	düğüm & ebeveyn karşıt taraflarda	düğüm iki kez döner

Splay ağacı: erişim



Bitti: 12 erişim. Splay ağacının garantisi kötümser değil, AMORTİZE: herhangi bir tek erişim $O(n)$ olabilir, ama m erişimin toplamı $O(m \log n)$ — ortalama $O(\log n)$.

Kod — splay()

```
void splay(Node *x) {  
    while (x->parent != NULL) {  
        Node *p = x->parent, *g = p->parent;  
        if (g == NULL) { rotate_up(x); }  
        else if ((x == p->left) == (p == g->left))  
            { rotate_up(p); rotate_up(x); }  
        else { rotate_up(x); rotate_up(x); }  
    }  
}
```

Beklenen çıktı

```
access(50): root = 50, inorder = 50  
access(20): root = 20, inorder = 10 20 30 40 45 50 60 70 80
```

Az önce erişilen anahtar HER ZAMAN yeni köktür.

Splay neden amortize $O(\log n)$

- Hiçbir tek erişim $O(\log n)$ garantili değil — $O(n)$ olabilir
- m erişimlik HERHANGİ bir dizi toplamda $O(m \log n)$ tutar
- Erişim başına amortize edilmiş $O(\log n)$
- "Sıcak" bir çalışma kümesine otomatik olarak uyum sağlar

Sık hata

- Zig-zig'i iki AYRI tekli döndürme olarak uygulamak ("saf splay")
- Geçerli bir ağaç işlemi, ama amortize garantiyi kaybeder
- `access` 'in OLMAYAN bir anahtarda da bir şeyi splay ettiğini unutmak
- Burada: yeni eklenen düğümün kendisi

Mini soru

Tek bir anahtara çok sayıda erişimden sonra, her DİĞER anahtar kabaca ne kadar derinde?

Mini yanıt

- O tek anahtar her seferinde kökte oturur
- Her DiĞER anahtarın derinliği neredeyse hiç etkilenmez
- Splay yalnızca erişilen YOL boyunca düğümleri yeniden düzenler
- "Küresel" bir denge garantisi yok — yalnızca yerel, erişim-başına

5. 2-3 Ağaçları

Başlangıç sorusu

Şimdiye kadarki her ağaç dengesizliği OLDUKTAN SONRA düzeltir. Ya dengesizlik yapısal olarak İMKANSIZ olsaydı?

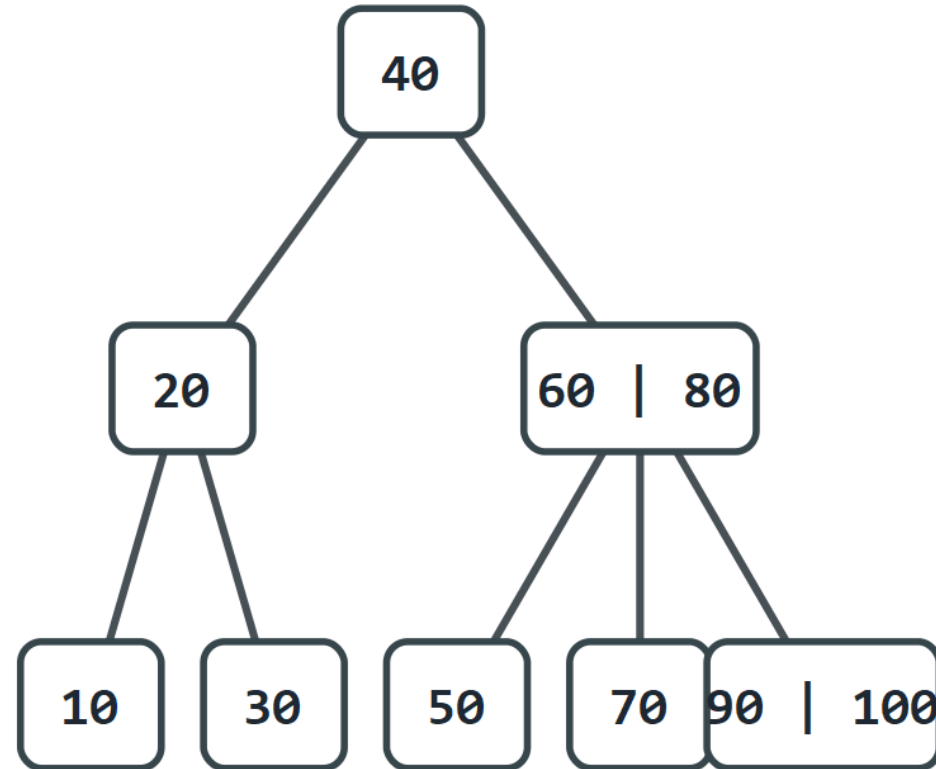
Kısa bir tarihçe

- **1972** — Rudolf Bayer & Edward McCreight
- B-ağacına (Hafta 14, disk-destekli dosyalar) bir ara adım
- Düğüm 1 anahtar (2-düğümü) ya da 2 anahtar (3-düğümü) tutar
- Her yaprak her zaman TAM OLARAK aynı derinliktedir

Fikir — taşma ve bölünme

- Yeni anahtar doğru yaprağa, sıralı olarak eklenir
- Yaprak zaten 2 anahtar tutuyorsa → şimdi geçici olarak 3: **taşma**
- İki 2-düğümüne bölünür; ORTA anahtar ebeveyne taşınır
- Ebeveyn aynı şekilde taşabilir — yukarı doğru katlanır

2-3 ağacı: ekleme



Bitti: 10 anahtar eklendi. Her yaprak hâlâ AYNI derinlikte — 2-3 ağacı asla dengesizleşmez, çünkü büyüme her zaman kökte olur.

Kod — taşma döngüsü

```
while (node->nkeys == 3) {
    split_node(node, &left, &right, &promoted);
    if (depth == 0) {
        /* kök bölünmesi: yükseklik + 1 */
        return new_root(promoted, left, right);
    }
    Node *parent = path[--depth];
    replace_with_split(parent, node,
                       promoted, left, right);
    node = parent;
}
```

Beklenen çıktı

```
insert(20): height = 0, level-order = [10,20]  
insert(30): height = 1, level-order = [20] [10] [30]
```

2-3 ağacı eklemesi neden $O(\log n)$

- Her yaprak aynı derinlikte $h \rightarrow h = O(\log n)$
- Aşağı yürüyüş: $O(h)$; bölünmeler en fazla $O(h)$ yukarı katlanır, her biri $O(1)$
- Bugünkü diğer her ağacın aksine, hiç döndürme yok
- Hafta 14'ün B-ağacının doğrudan atası

Sık hata

- `child[]` 'i yalnız 3 yuvaya boyutlandırmak (kararlı-durum maksimumu)
- Taşma ortasında, bir düğüm geçici olarak 4 çocuğa ihtiyaç duyar — gerçek tampon taşması
- Bir düğümü böldükten sonra eski kabuğu serbest bırakmamak
- Yanlış anahtarı taşımak (ORTA olmalı)

Mini soru

Bir 2-3 ağacının yüksekliği neden YALNIZ kökte büyür,
hiç ortada bir yerde değil?

Mini yanıt

- Bir bölünme yalnız KENDİ düğümünün taşmasına yanıt verir
- Taşınan anahtar KENDİ ebeveynine gider, asla bir kardeşe değil
- Katlanma yalnız bir yol boyunca dümdüz yukarı gidebilir
- "Ebeveyni" tükenebileceği tek yer köktür

6. Segment Ağaçları

Başlangıç sorusu

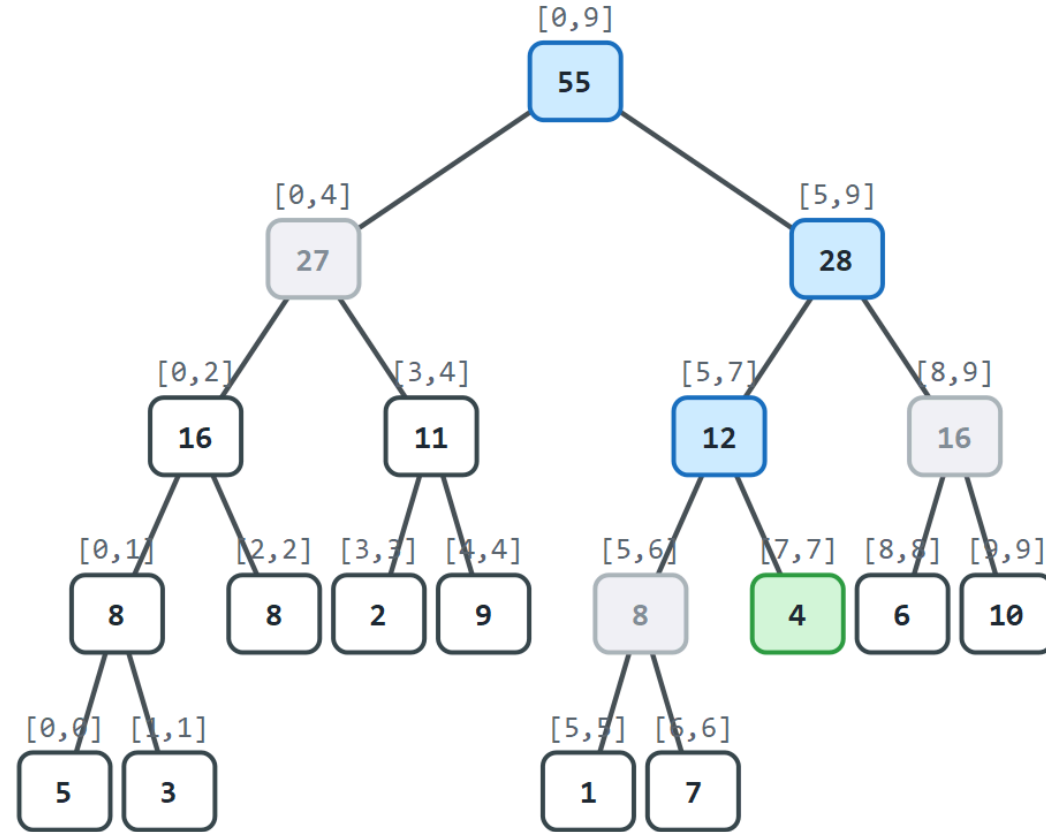
"l ile r arasındaki her değerin TOPLAMI nedir?"

Bir döngü $O(n)$ 'de yanıtlar. Birçok böyle sorgu — daha iyisi mümkün mü?

Fikir — her aralığı bir kez önceden hesapla

- Sabit boyutlu bir dizi üzerinde bir kez kurulur
- Düğüm i , $[l_i, h_i]$ aralığından sorumlu; çocuklar $2i$, $2i+1$
- Yaprak bir değer tutar; iç düğüm iki çocuğunun toplamını tutar
- Sorgu aşağı yürür: dışarı $\rightarrow 0$, içeri $\rightarrow$ önceden hesaplanmış, kısmi $\rightarrow$ ikisine de in

Segment ağacı: kurulum ve sorgu



Bitti: 3 sorgu. Her sorgu en fazla $O(\log n)$ düğüm ziyaret eder — bir düz döngünün $O(n)$ 'ine karşı büyük kazanç, özellikle çok sayıda sorgu varsa.

Kod — query()

```
long query(int i, int lo, int hi, int l, int r) {  
    if (r < lo || hi < l)    return 0;  
    if (l <= lo && hi <= r) return tree[i];  
    int mid = (lo + hi) / 2;  
    return query(2*i,    lo,    mid, l, r)  
        + query(2*i+1, mid + 1, hi,  l, r);  
}
```

Beklenen çıktı

```
query(0,9) = 55  
query(2,5) = 20  
query(7,7) = 4
```

Segment ağacı sorgusu neden $O(\log n)$

- build(): toplamda $O(n)$ — her düğümü bir kez ziyaret eder
- Her seviye: en fazla İKİ "kısmen kesişen" düğüm
- O seviyedeki her diğer düğüm: hemen yanıtlanır ya da budanır
- Toplam iş yükseklikle orantılı: $O(\log n)$

Bölüm 1–5'ten yapısal bir fark

- Segment ağacı BİÇİMİ yalnız n'e bağlıdır, hiç veri değerine değil
- Her BST-ailesi ağacının biçimi DEĞER karşılaştırmalarına bağlıdır
- Bir segment ağacı bölüm 1.4'ün BST'si gibi asla dejenere olamaz
- Burada hiç ekleme-sırası problemi yok

Sık hata

- Diziyi $4n$ yerine $2n$ olarak boyutlandırmak
- "Tamamen içeride"yi "tamamen dışarıda" ile karıştırmak (ters koşullar)
- Tek bir nokta güncellemesi için BÜTÜN ağacı ($O(n)$) yeniden kurmak
- Özel bir $O(\log n)$ nokta-güncelleme fonksiyonu doğal düzeltmedir

Mini soru

Bir sorgunun maliyeti neden $O(\log n)$ 'dir, her seviyedeki düğüm sayısı ÇARPI $O(\log n)$ değil?

Mini yanıt

- Her seviyede en fazla İKİ düğüm "kısmen kesişiyor"
- Her diğer düğüm: tamamen içeride (bitti) ya da tamamen dışarıda (budandı)
- Toplam iş genişlikle değil yükseklikle orantılı
- $O(\log n)$, $O(\log n) \cdot O(n)$ değil

7. Fenwick Ağaçları

Başlangıç sorusu

Bir segment ağacı en fazla $4n$ açık düğüme ihtiyaç duyar.
DÜZ BİR DİZİ önek toplamalarını aynı hızda yanıtlayabilir mi?

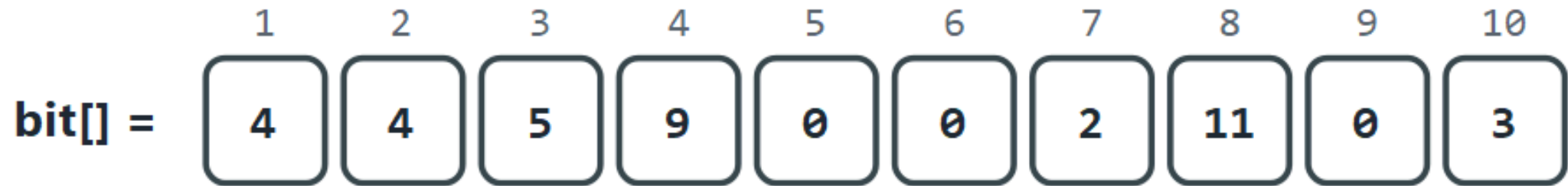
Kısa bir tarihçe

- **1994** — Peter Fenwick
- "Kümülatif frekans tabloları için yeni bir veri yapısı"
- Ağaç göstericisi yok, özyineleme gerekmez
- Tek bir dizi, tek bir aritmetik hile: `i & -i`

Fikir — i & $-i$ en düşük ayarlı biti yalıtır

- `bit[i]`, `i` 'de biten `i & -i` büyüklüğünde bir aralığın toplamını tutar
- `update(i, delta)`: YUKARI yürü, `i += i & -i`
- `query(i)`: önek toplamı 1..i, AŞAĞI yürü, `i -= i & -i`
- İki yürüyüş de: $O(\log n)$ adım, ağaç yapısı gerekmez

Fenwick ağacı: güncelleme ve sorgu



Bitti: 8 işlem. Her update ve her query en fazla $O(\log n)$ adım sürer — i & $-i$, ikilik (binary) tabanda en sağdaki 1 bitini yalıtın basit bir aritmetik hile.

i & -i zincir uzunluğu, tek başına



Bitti: 2 işlem. Her update ve her query en fazla $O(\log n)$ adım sürer — i & -i, ikilik (binary) tabanda en sağdaki 1 bitini yalıtın basit bir aritmetik hile.

Kod — update() ve query()

```
void update(int i, int delta) {  
    while (i <= n) { bit[i] += delta; i += i & (-i); }  
}  
int query(int i) {  
    int sum = 0;  
    while (i > 0) { sum += bit[i]; i -= i & (-i); }  
    return sum;  
}
```

Beklenen çıktı

```
update(3, 5)  
update(7, 2)  
query(10) = 7
```

Fenwick neden $O(\log n)$, $O(n)$ alan

- i & $-i$, sınıra olan mesafeyi en az iki katına (update) ya da yarıya (query) çıkarır
- Her iki yönde de en fazla $\text{floor}(\log_2(n)) + 1$ yineleme
- Alan: düz bir int dizisi — bir segment ağacından çarpıcı ölçüde az
- Dizin boyutu değişmeyecekse pratikte tercih edilir

Sık hata

- 0-indisleme kullanmak — `i & -i`, indis 0'ın tüm sıfır bit olmasını ister
- Fenwick ağaçları HER ZAMAN 1-indislidir
- Dilin `-`'si yerine kendi yapımı bir "negasyon" yazmak
- Aralık-MİN/MAKS sorguları için Fenwick'e başvurmak (çalışmaz)

Mini soru

$n=16$ için, `update(1)` neden tam olarak 5 adım tutar
($i = 1, 2, 4, 8, 16$)?

Mini yanıt

- `i += i & -i` : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$, sonra döngü durur (i, n'i geçer)
- Beş hücreye dokunulur, her biri indis 1'i içeren bir aralıktan sorumlu
- `query(15)` bunun yerine 4 adım tutar: 15'in ikili gösterimindeki (1111) her 1-biti için bir tane
- Güncelleme ve sorgu karşıt yönlerde yürür, farklı bit örüntüleri

8. Teknik Seçme

Karşılaştırma — dengeli BST'ler

Yapı	En kötü durum yüksekliği	Yeniden dengeleme maliyeti
Düz BST	$O(n)$	yok
AVL	$O(\log n)$, en sıkı	eklemede ≤ 1 döndürme
Kırmızı-siyah	$O(\log n)$, daha gevşek	ortalamada daha az döndürme
Splay	Amortize $O(\log n)$	erişim başına tam splay

Karşılaştırma — yapısal & aralık ağaçları

Yapı	Yeniden dengeleme maliyeti	En iyi olduğu yer
2-3 ağacı	düğüm bölünmeleri, döndürme yok	Hafta 14'ün B-ağacına köprü
Segment ağacı	kurulumdan sonra yok	çok sayıda aralık toplamı/min/maks
Fenwick ağacı	yok	aralık toplamı + sık nokta güncellemesi

Seçme — iş yüküne göre

- Arama-ağırlıklı → AVL (en sıkı sınır)
- Karışık ekleme/silme/arama → kırmızı-siyah (daha az döndürme)
- Eğik "sıcak anahtar" erişimi → splay (uyum sağlar, düşük bellek)
- Aralık soruları → segment ağacı ya da Fenwick ağacı

Mini soru

Bir meslektaşınız "her zaman kırmızı-siyah kullan, en dengeli ve kütüphane-test edilmiş" diyor. Önce ne sorardınız?

Mini yanıt

- İş yükü arama-ağırlıklı mı, yoksa karışık ekleme/silme/arama mı?
- Erişim küçük bir sıcak-anahtar kümesine doğru eğik mi?
- Sorular tek anahtarlar değil ARALIKLAR hakkında mı?
- Bu daha sonra disk-destekli bir yapıya mı besleniyor (Hafta 14)?

Özet

- BST: `insert` / `search` / `delete` hepsi $O(h)$ — ama h SIRAYA bağlı
- Sıralı girdi bir BST'yi bir zincire dejenere eder: $O(n)$, bir listeden iyi değil
- AVL, kırmızı-siyah, splay, 2-3 ağacı: dört farklı düzeltme, dört ödünleşim
- Segment ağacı, Fenwick ağacı: ARALIK sorularına $O(\log n)$ yanıtlar

Alıştırmalar önizlemesi

- h yüksekliğindeki bir BST'nin en fazla $2^{(h+1)} - 1$ düğümü olduğunu kanıtlayın
- Kırmızı-siyahın fixup durumlarını "zor" senaryoda elle izleyin
- Bir segment ağacını aralık-MİN'e çevirmek için nelerin değişeceğini taslak çizin
- 10 alıştırmanın tam listesi: bu haftanın notları

Kendini sınama testi önizlemesi

- AVL ekleme neden en fazla bir döndürme ister, ama silme neden katlanabilir?
- Bir Fenwick ağacı aralık-minimum sorgularını neden desteklemez?
- Bir segment ağacının biçimi veriden neden bağımsızdır?
- Yanıtlarıyla tam 10 soruluk test: bu haftanın notları

İleriye bakış

- **Hafta 12** — dizgeler: eşleştirme algoritmaları, ve **trie**
- Bir trie, dizgeleri sayısal karşılaştırmayla değil paylaşılan ÖNEKLE saklar
- **Hafta 13** — doğrudan ve sıralı dosya organizasyonu
- **Hafta 14** — **B-ağacı**: bu haftanın 2-3 ağacı, disk için genellenmiş

Sorular?

CEN207 Veri Yapıları — Hafta 11

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz